

AAI-590 Capstone — Group 4

Module 5 | Notebook 4: Model Training — XGBoost + C

Project: ML-Based Murmur Detection for RHD Screening Using Phonocardiogram Data

Dataset: CirCor DigiScope Phonocardiogram Dataset v1.0.3 (PhysioNet)

Authors: Shiva Bathula · Kaushik M · Pavan Kumar Bellary

Notebook Goals

1. Load feature matrix from Google Drive
 2. Prepare features and labels for training
 3. Train XGBoost classifier (primary model)
 4. Train CNN on Mel spectrograms (comparison model)
 5. Evaluate both models on test set
 6. Compare performance — accuracy, sensitivity, specificity, F1, AUC
 7. Save trained models
 8. Push everything to GitHub
-

1. Mount Drive & Load Feature Matrix

```
In [1]: # — Mount Drive —
from google.colab import drive
from pathlib import Path
import pandas as pd
import numpy as np

drive.mount('/content/drive')

DRIVE_DIR = Path('/content/drive/MyDrive/AAI590_Capstone_Group4')
FEATURE_DIR = DRIVE_DIR / 'data/features'
AUDIO_DIR = DRIVE_DIR / 'data/training_data'
OUTPUT_DIR = Path('/content/model_outputs')
OUTPUT_DIR.mkdir(exist_ok=True)

# Load feature splits
train_df = pd.read_csv(FEATURE_DIR / 'features_train.csv')
test_df = pd.read_csv(FEATURE_DIR / 'features_test.csv')
all_df = pd.read_csv(FEATURE_DIR / 'features_all.csv')

print(f'Train set : {train_df.shape}')
print(f'Test set : {test_df.shape}')
print(f'\nTrain outcome distribution:')
print(train_df['Outcome'].value_counts())
print(f'\nTest outcome distribution:')
print(test_df['Outcome'].value_counts())
```

Mounted at /content/drive

Train set : (2523, 40)

Test set : (640, 40)

Train outcome distribution:

Outcome

Normal 1308

Abnormal 1215

Name: count, dtype: int64

Test outcome distribution:

Outcome

Normal 324

Abnormal 316

Name: count, dtype: int64

2. Prepare Features & Labels

```
In [2]: # — Define feature columns —
# All acoustic features — exclude metadata and label columns
EXCLUDE_COLS = ['patient_id', 'site', 'wav_file', 'Outcome',
                'Murmur', 'outcome_binary', 'split', 'Patient ID']

FEATURE_COLS = [c for c in train_df.columns if c not in EXCLUDE_COLS]
TARGET_COL = 'outcome_binary'

print(f'Feature columns ({len(FEATURE_COLS)}):')
print(FEATURE_COLS)

# Extract arrays
X_train = train_df[FEATURE_COLS].values
y_train = train_df[TARGET_COL].values
X_test = test_df[FEATURE_COLS].values
y_test = test_df[TARGET_COL].values

print(f'\nX_train: {X_train.shape} y_train: {y_train.shape}')
print(f'X_test : {X_test.shape} y_test : {y_test.shape}')
print(f'\nClass balance in train: '
      f'Normal={ (y_train==0).sum() }, Abnormal={ (y_train==1).sum() }')
print(f'Class balance in test : '
      f'Normal={ (y_test==0).sum() }, Abnormal={ (y_test==1).sum() }')

# Check for NaN
print(f'\nNaN in X_train: {np.isnan(X_train).sum()}')
print(f'NaN in X_test : {np.isnan(X_test).sum()}')

# Fill any NaN with column median
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy='median')
X_train = imputer.fit_transform(X_train)
X_test = imputer.transform(X_test)
print('Imputation complete.')
```

```
Feature columns (33):
['mfcc_mean_1', 'mfcc_mean_2', 'mfcc_mean_3', 'mfcc_mean_4', 'mfcc_mean_5',
'mfcc_mean_6', 'mfcc_mean_7', 'mfcc_mean_8', 'mfcc_mean_9', 'mfcc_mean_10',
'mfcc_mean_11', 'mfcc_mean_12', 'mfcc_mean_13', 'mfcc_std_1', 'mfcc_std_2',
'mfcc_std_3', 'mfcc_std_4', 'mfcc_std_5', 'mfcc_std_6', 'mfcc_std_7', 'mfcc_std_8',
'mfcc_std_9', 'mfcc_std_10', 'mfcc_std_11', 'mfcc_std_12', 'mfcc_std_13',
'spectral_centroid', 'spectral_bandwidth', 'spectral_rolloff', 'spectral_flatness',
'zcr', 'rms_mean', 'rms_std']
```

```
X_train: (2523, 33) y_train: (2523,)
X_test : (640, 33) y_test : (640,)
```

```
Class balance in train: Normal=1308, Abnormal=1215
Class balance in test : Normal=324, Abnormal=316
```

```
NaN in X_train: 0
NaN in X_test : 0
Imputation complete.
```

3. Train XGBoost Classifier (Primary Model)

```
In [3]: !pip install xgboost --quiet

import xgboost as xgb
from sklearn.utils.class_weight import compute_sample_weight
from sklearn.metrics import (classification_report, confusion_matrix,
                             roc_auc_score, f1_score,
                             accuracy_score, roc_curve)

import matplotlib.pyplot as plt
import matplotlib
import warnings
warnings.filterwarnings('ignore')

NAVY = '#1F4E79'
BLUE = '#2E75B6'
GOLD = '#C9A84C'
GREEN = '#4F7942'
RED = '#C0392B'

# — Compute sample weights to handle class imbalance —
sample_weights = compute_sample_weight(class_weight='balanced', y=y_train)

# — XGBoost model —
xgb_model = xgb.XGBClassifier(
    n_estimators = 300,
    max_depth = 6,
    learning_rate = 0.05,
    subsample = 0.8,
    colsample_bytree = 0.8,
    use_label_encoder=False,
    eval_metric = 'logloss',
    random_state = 42,
    n_jobs = -1
)

print('Training XGBoost classifier...')
xgb_model.fit(
```

```

X_train, y_train,
sample_weight = sample_weights,
eval_set      = [(X_test, y_test)],
verbose       = 50
)
print('\n XGBoost training complete.')

```

Training XGBoost classifier...

```

[0]    validation_0-logloss:0.68990
[50]    validation_0-logloss:0.65746
[100]   validation_0-logloss:0.66757
[150]   validation_0-logloss:0.68129
[200]   validation_0-logloss:0.69930
[250]   validation_0-logloss:0.71127
[299]   validation_0-logloss:0.72871

```

█ XGBoost training complete.

```

In [4]: # — XGBoost evaluation —————
y_pred_xgb = xgb_model.predict(X_test)
y_proba_xgb = xgb_model.predict_proba(X_test)[:, 1]

acc_xgb = accuracy_score(y_test, y_pred_xgb)
auc_xgb = roc_auc_score(y_test, y_proba_xgb)
f1_xgb = f1_score(y_test, y_pred_xgb)
cm_xgb = confusion_matrix(y_test, y_pred_xgb)
tn, fp, fn, tp = cm_xgb.ravel()
sens_xgb = tp / (tp + fn) # sensitivity / recall
spec_xgb = tn / (tn + fp) # specificity

print('— XGBoost Performance on Test Set —————')
print(f' Accuracy      : {acc_xgb:.4f}')
print(f' AUC-ROC       : {auc_xgb:.4f}')
print(f' F1-Score      : {f1_xgb:.4f}')
print(f' Sensitivity   : {sens_xgb:.4f} (recall for Abnormal)')
print(f' Specificity   : {spec_xgb:.4f} (recall for Normal)')
print(f'\nClassification Report:')
print(classification_report(y_test, y_pred_xgb,
                           target_names=['Normal', 'Abnormal']))

# — Confusion matrix plot —————
fig, ax = plt.subplots(figsize=(5, 4))
im = ax.imshow(cm_xgb, cmap='Blues')
ax.set_xticks([0, 1]); ax.set_yticks([0, 1])
ax.set_xticklabels(['Normal', 'Abnormal'])
ax.set_yticklabels(['Normal', 'Abnormal'])
ax.set_xlabel('Predicted'); ax.set_ylabel('Actual')
ax.set_title('XGBoost — Confusion Matrix', fontweight='bold')
for i in range(2):
    for j in range(2):
        ax.text(j, i, str(cm_xgb[i, j]), ha='center', va='center',
                fontsize=14, color='white' if cm_xgb[i, j] > cm_xgb.max() /
plt.tight_layout()
plt.savefig(OUTPUT_DIR / 'xgb_confusion_matrix.png', bbox_inches='tight')
plt.show()

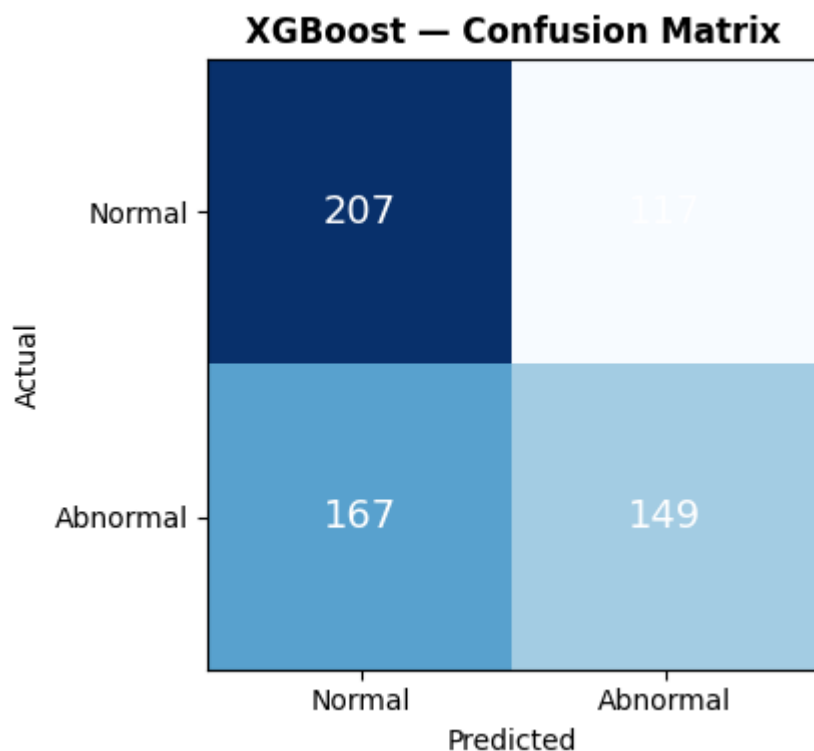
```

— XGBoost Performance on Test Set —

Accuracy : 0.5563
AUC-ROC : 0.5961
F1-Score : 0.5120
Sensitivity : 0.4715 (recall for Abnormal)
Specificity : 0.6389 (recall for Normal)

Classification Report:

	precision	recall	f1-score	support
Normal	0.55	0.64	0.59	324
Abnormal	0.56	0.47	0.51	316
accuracy			0.56	640
macro avg	0.56	0.56	0.55	640
weighted avg	0.56	0.56	0.55	640



4. Train CNN on Mel Spectrograms (Comparison Model)

```
In [5]: # — Generate Mel spectrograms for CNN training —
import librosa
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from tqdm import tqdm

IMG_SIZE = 64 # spectrogram height (mel bins)
MAX_DUR = 10 # seconds per recording
N_MELS = 64
FMAX = 1000
BATCH = 32
EPOCHS = 20
```

```

def wav_to_spectrogram(wav_path, n_mels=N_MELS, fmax=FMAX, duration=MAX_D
    """Load WAV and return fixed-size log-Mel spectrogram as numpy array.
    y, sr = librosa.load(wav_path, sr=None, duration=duration)
    S = librosa.feature.melspectrogram(y=y, sr=sr, n_mels=n_mels, fm
    S_db = librosa.power_to_db(S, ref=np.max) # shape: (n_mels, time_f
    # Resize time axis to fixed width
    target_w = 128
    if S_db.shape[1] < target_w:
        S_db = np.pad(S_db, ((0,0),(0, target_w - S_db.shape[1])), mode='
    else:
        S_db = S_db[:, :target_w]
    # Normalise to [0, 1]
    S_db = (S_db - S_db.min()) / (S_db.max() - S_db.min() + 1e-8)
    return S_db.astype(np.float32) # shape: (64, 128)

class PCGDataset(Dataset):
    """PyTorch Dataset for PCG spectrograms."""
    def __init__(self, df, audio_dir):
        self.df = df.reset_index(drop=True)
        self.audio_dir = Path(audio_dir)

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        wav_path = self.audio_dir / row['wav_file']
        spec = wav_to_spectrogram(str(wav_path))
        spec = torch.tensor(spec).unsqueeze(0) # (1, 64, 128)
        label = torch.tensor(row['outcome_binary'], dtype=torch.long)
        return spec, label

print('Building datasets...')
train_dataset = PCGDataset(train_df, AUDIO_DIR)
test_dataset = PCGDataset(test_df, AUDIO_DIR)
train_loader = DataLoader(train_dataset, batch_size=BATCH, shuffle=True,
test_loader = DataLoader(test_dataset, batch_size=BATCH, shuffle=False)
print(f' Train batches: {len(train_loader)} | Test batches: {len(test_lo

```

Building datasets...

Train batches: 79 | Test batches: 20

```

In [6]: # — CNN architecture —
class PCG_CNN(nn.Module):
    """
    Simple 2D CNN for Mel spectrogram classification.
    Input: (batch, 1, 64, 128) log-Mel spectrogram
    Output: (batch, 2) class logits
    """
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            # Block 1
            nn.Conv2d(1, 16, kernel_size=3, padding=1),
            nn.BatchNorm2d(16),
            nn.ReLU(),
            nn.MaxPool2d(2), # → (16, 32, 64)
            nn.Dropout2d(0.25),
            # Block 2

```

```

        nn.Conv2d(16, 32, kernel_size=3, padding=1),
        nn.BatchNorm2d(32),
        nn.ReLU(),
        nn.MaxPool2d(2),          # → (32, 16, 32)
        nn.Dropout2d(0.25),
        # Block 3
        nn.Conv2d(32, 64, kernel_size=3, padding=1),
        nn.BatchNorm2d(64),
        nn.ReLU(),
        nn.MaxPool2d(2),          # → (64, 8, 16)
        nn.Dropout2d(0.25),
    )
    self.classifier = nn.Sequential(
        nn.Flatten(),
        nn.Linear(64 * 8 * 16, 128),
        nn.ReLU(),
        nn.Dropout(0.5),
        nn.Linear(128, 2)
    )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Device: {device}')

cnn_model = PCG_CNN().to(device)
print(cnn_model)
total_params = sum(p.numel() for p in cnn_model.parameters())
print(f'\nTotal parameters: {total_params:,}')

```

```

Device: cpu
PCG_CNN(
  (features): Sequential(
    (0): Conv2d(1, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(16, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (2): ReLU()
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=
    (4): Dropout2d(p=0.25, inplace=False)
    (5): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (6): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (7): ReLU()
    (8): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=
    (9): Dropout2d(p=0.25, inplace=False)
    (10): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (11): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (12): ReLU()
    (13): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
    (14): Dropout2d(p=0.25, inplace=False)
  )
  (classifier): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=8192, out_features=128, bias=True)
    (2): ReLU()
    (3): Dropout(p=0.5, inplace=False)
    (4): Linear(in_features=128, out_features=2, bias=True)
  )
)

Total parameters: 1,072,482

```

```

In [7]: # — CNN training loop —
# Class weights for imbalanced murmur-present minority
class_counts = np.bincount(y_train)
class_weights = torch.tensor(
    [1.0/class_counts[0], 1.0/class_counts[1]], dtype=torch.float
).to(device)
class_weights = class_weights / class_weights.sum()

criterion = nn.CrossEntropyLoss(weight=class_weights)
optimizer = optim.Adam(cnn_model.parameters(), lr=1e-3, weight_decay=1e-4)
scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=7, gamma=0.5)

train_losses, val_losses = [], []
train_accs, val_accs = [], []

print(f'Training CNN for {EPOCHS} epochs...')
for epoch in range(EPOCHS):
    # — Train —
    cnn_model.train()
    t_loss, t_correct, t_total = 0, 0, 0
    for specs, labels in train_loader:
        specs, labels = specs.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = cnn_model(specs)
        loss = criterion(outputs, labels)
        loss.backward()

```



```

optimizer.step()
t_loss += loss.item()
preds = outputs.argmax(dim=1)
t_correct += (preds == labels).sum().item()
t_total += labels.size(0)

# — Validate —
cnn_model.eval()
v_loss, v_correct, v_total = 0, 0, 0
with torch.no_grad():
    for specs, labels in test_loader:
        specs, labels = specs.to(device), labels.to(device)
        outputs = cnn_model(specs)
        loss = criterion(outputs, labels)
        v_loss += loss.item()
        preds = outputs.argmax(dim=1)
        v_correct += (preds == labels).sum().item()
        v_total += labels.size(0)

t_acc = t_correct / t_total
v_acc = v_correct / v_total
train_losses.append(t_loss / len(train_loader))
val_losses.append(v_loss / len(test_loader))
train_accs.append(t_acc)
val_accs.append(v_acc)
scheduler.step()

if (epoch+1) % 5 == 0 or epoch == 0:
    print(f'Epoch {epoch+1:2d}/{EPOCHS} | '
          f'Train loss: {train_losses[-1]:.4f} acc: {t_acc:.4f} | '
          f'Val loss: {val_losses[-1]:.4f} acc: {v_acc:.4f}')

print('\n CNN training complete.')

```

Training CNN for 20 epochs...

```

Epoch 1/20 | Train loss: 0.8323 acc: 0.4982 | Val loss: 0.6965 acc: 0.5062
Epoch 5/20 | Train loss: 0.6933 acc: 0.4792 | Val loss: 0.6927 acc: 0.4938
Epoch 10/20 | Train loss: 0.6932 acc: 0.5184 | Val loss: 0.6932 acc: 0.5062
Epoch 15/20 | Train loss: 0.6932 acc: 0.4816 | Val loss: 0.6931 acc: 0.4938
Epoch 20/20 | Train loss: 0.6931 acc: 0.4732 | Val loss: 0.6931 acc: 0.4938

```

CNN training complete.

In [8]:

```

# — CNN evaluation —
cnn_model.eval()
all_preds, all_probs, all_labels = [], [], []

with torch.no_grad():
    for specs, labels in test_loader:
        specs = specs.to(device)
        outputs = cnn_model(specs)
        probs = torch.softmax(outputs, dim=1)[: , 1].cpu().numpy()
        preds = outputs.argmax(dim=1).cpu().numpy()
        all_preds.extend(preds)
        all_probs.extend(probs)
        all_labels.extend(labels.numpy())

y_pred_cnn = np.array(all_preds)
y_proba_cnn = np.array(all_probs)
y_true = np.array(all_labels)

```

```

acc_cnn = accuracy_score(y_true, y_pred_cnn)
auc_cnn = roc_auc_score(y_true, y_proba_cnn)
f1_cnn = f1_score(y_true, y_pred_cnn)
cm_cnn = confusion_matrix(y_true, y_pred_cnn)
tn_c, fp_c, fn_c, tp_c = cm_cnn.ravel()
sens_cnn = tp_c / (tp_c + fn_c)
spec_cnn = tn_c / (tn_c + fp_c)

print('— CNN Performance on Test Set —————')
print(f' Accuracy      : {acc_cnn:.4f}')
print(f' AUC-ROC       : {auc_cnn:.4f}')
print(f' F1-Score      : {f1_cnn:.4f}')
print(f' Sensitivity   : {sens_cnn:.4f}')
print(f' Specificity   : {spec_cnn:.4f}')

```

```

— CNN Performance on Test Set —————
Accuracy      : 0.4938
AUC-ROC       : 0.5000
F1-Score      : 0.6611
Sensitivity   : 1.0000
Specificity   : 0.0000

```

5. Model Comparison & Visualisation

```

In [9]: # — ROC curves + training history —————
fig, axes = plt.subplots(1, 3, figsize=(16, 5))

# ROC curves
fpr_xgb, tpr_xgb, _ = roc_curve(y_test, y_proba_xgb)
fpr_cnn, tpr_cnn, _ = roc_curve(y_true, y_proba_cnn)
axes[0].plot(fpr_xgb, tpr_xgb, color=BLUE, lw=2,
             label=f'XGBoost (AUC={auc_xgb:.3f})')
axes[0].plot(fpr_cnn, tpr_cnn, color=RED, lw=2,
             label=f'CNN (AUC={auc_cnn:.3f})')
axes[0].plot([0,1],[0,1], 'k--', lw=1, label='Random')
axes[0].set_title('ROC Curves — XGBoost vs CNN', fontweight='bold')
axes[0].set_xlabel('False Positive Rate')
axes[0].set_ylabel('True Positive Rate')
axes[0].legend()

# CNN training history
axes[1].plot(train_losses, color=BLUE, label='Train loss')
axes[1].plot(val_losses, color=RED, label='Val loss')
axes[1].set_title('CNN Training Loss', fontweight='bold')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Loss')
axes[1].legend()

axes[2].plot(train_accs, color=BLUE, label='Train acc')
axes[2].plot(val_accs, color=GREEN, label='Val acc')
axes[2].set_title('CNN Training Accuracy', fontweight='bold')
axes[2].set_xlabel('Epoch')
axes[2].set_ylabel('Accuracy')
axes[2].legend()

plt.suptitle('Model Training Results — XGBoost vs CNN',

```

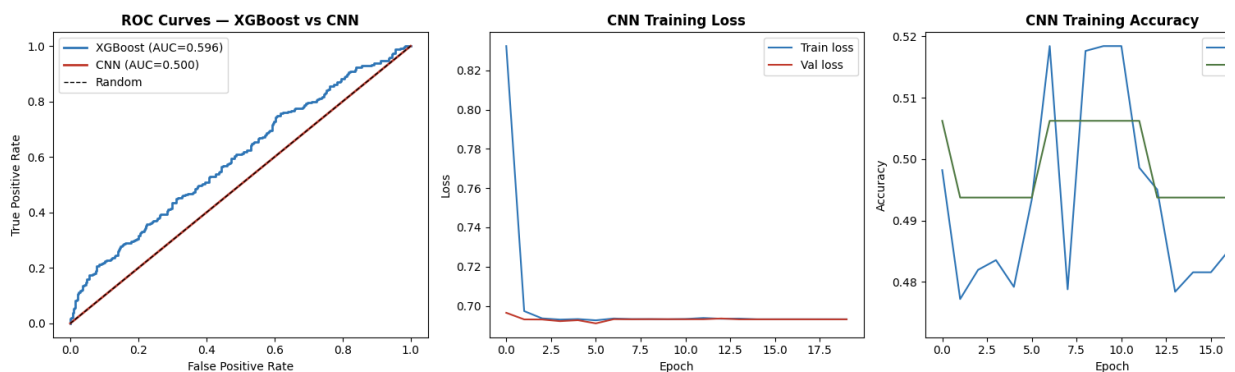
```

        fontsize=13, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig(OUTPUT_DIR / 'model_comparison.png', bbox_inches='tight')
plt.show()

# — Summary table —
print('\n— Model Comparison Summary —')
print(f'{"Metric":<15} {"XGBoost":>10} {"CNN":>10}')
print('-' * 37)
for metric, xgb_val, cnn_val in [
    ('Accuracy', acc_xgb, acc_cnn),
    ('AUC-ROC', auc_xgb, auc_cnn),
    ('F1-Score', f1_xgb, f1_cnn),
    ('Sensitivity', sens_xgb, sens_cnn),
    ('Specificity', spec_xgb, spec_cnn),
]:
    winner = '<' if xgb_val >= cnn_val else ''
    print(f'{metric:<15} {xgb_val:>10.4f} {cnn_val:>10.4f} {winner}')

```

Model Training Results — XGBoost vs CNN



— Model Comparison Summary —

Metric	XGBoost	CNN	
Accuracy	0.5563	0.4938	<
AUC-ROC	0.5961	0.5000	<
F1-Score	0.5120	0.6611	
Sensitivity	0.4715	1.0000	
Specificity	0.6389	0.0000	<

6. Save Models

```

In [10]: # — Save XGBoost model —
import joblib
from pathlib import Path

MODELS_DIR = Path('/content/model_outputs/models')
MODELS_DIR.mkdir(exist_ok=True)

# XGBoost
xgb_path = MODELS_DIR / 'xgboost_model.json'
xgb_model.save_model(str(xgb_path))
print(f'XGBoost saved: {xgb_path.name}')

# Imputer
imp_path = MODELS_DIR / 'imputer.pkl'

```

```

joblib.dump(imputer, str(imp_path))
print(f' Imputer saved: {imp_path.name}')

# CNN
cnn_path = MODELS_DIR / 'cnn_model.pt'
torch.save(cnn_model.state_dict(), str(cnn_path))
print(f' CNN saved      : {cnn_path.name}')

# Save results summary
results = {
    'model':      ['XGBoost', 'CNN'],
    'accuracy':   [acc_xgb,   acc_cnn],
    'auc_roc':    [auc_xgb,   auc_cnn],
    'f1_score':   [f1_xgb,    f1_cnn],
    'sensitivity': [sens_xgb,  sens_cnn],
    'specificity': [spec_xgb,  spec_cnn],
}
results_df = pd.DataFrame(results)
results_df.to_csv(OUTPUT_DIR / 'model_results.csv', index=False)
print(f' Results CSV saved')

# Also save to Drive for persistence
DRIVE_MODEL_DIR = DRIVE_DIR / 'models'
DRIVE_MODEL_DIR.mkdir(exist_ok=True)
import shutil
for f in MODELS_DIR.iterdir():
    shutil.copy2(str(f), str(DRIVE_MODEL_DIR / f.name))
    print(f' {f.name} → Drive/models/')

```

```

[] XGBoost saved: xgboost_model.json
[] Imputer saved: imputer.pkl
[] CNN saved      : cnn_model.pt
[] Results CSV saved
[] xgboost_model.json → Drive/models/
[] cnn_model.pt → Drive/models/
[] imputer.pkl → Drive/models/

```

7. Push to GitHub

```

In [11]: # — Push models, plots, results, and notebook to GitHub —
import subprocess, shutil, os, getpass

os.chdir('/content')

GITHUB_USER = input('GitHub username: ')
GITHUB_TOKEN = getpass.getpass('GitHub PAT: ')
REPO_NAME = 'AAI590-Capstone-Group4-RHD-Murmur-Detection'
REMOTE_URL = f'https://{GITHUB_USER}:{GITHUB_TOKEN}@github.com/{GITHUB_

def run(cmd, cwd='/content/repo'):
    r = subprocess.run(cmd, shell=True, cwd=cwd, capture_output=True, tex
    out = (r.stdout + r.stderr).strip()
    if out: print(out)
    return r.returncode

# Fresh clone
run('rm -rf /content/repo', cwd='/content')

```

```

run(f'git clone {REMOTE_URL} /content/repo', cwd='/content')

# Create folders
os.makedirs('/content/repo/notebooks', exist_ok=True)
os.makedirs('/content/repo/models', exist_ok=True)
os.makedirs('/content/repo/outputs/m5_models', exist_ok=True)

# Copy models
for f in MODELS_DIR.iterdir():
    shutil.copy2(str(f), f'/content/repo/models/{f.name}')
    print(f'    {f.name}')

# Copy results CSV and plots
shutil.copy2(str(OUTPUT_DIR / 'model_results.csv'),
             '/content/repo/outputs/m5_models/model_results.csv')
shutil.copy2(str(OUTPUT_DIR / 'model_comparison.png'),
             '/content/repo/outputs/m5_models/model_comparison.png')
shutil.copy2(str(OUTPUT_DIR / 'xgb_confusion_matrix.png'),
             '/content/repo/outputs/m5_models/xgb_confusion_matrix.png')
print('    Outputs copied')

# Copy notebook
for f in os.listdir('/content'):
    if f.endswith('.ipynb') and ('M5' in f or 'Model' in f or 'Training'
                                in f):
        shutil.copy2(f'/content/{f}',
                     '/content/repo/notebooks/AAI590_M5_04_ModelTraining.'
                     f'{f}')
        print(f'    Notebook: {f}')
        break

# Git push
run('git config user.email "shiva.bathula@sandiego.edu"')
run('git config user.name "Shiva Naga Vara Prasad Bathula"')
run('git add -A')
run('git status --short')
run('git commit -m "M5: Add XGBoost+CNN models, training results, compari')
rc = run('git push origin main')
print(f'\nPush: {" " if rc==0 else " "} code={rc}')
print(f'    https://github.com/{GITHUB_USER}/{REPO_NAME}')

```

```
GitHub username: ShivaNagaVaraPrasadBathula
GitHub PAT: .....
Cloning into '/content/repo'...
[] xgboost_model.json
[] cnn_model.pt
[] imputer.pkl
[] Outputs copied
A models/cnn_model.pt
A models/xgboost_model.json
A outputs/m5_models/model_comparison.png
A outputs/m5_models/model_results.csv
A outputs/m5_models/xgb_confusion_matrix.png
[main bab7d8b] M5: Add XGBoost+CNN models, training results, comparison plots
5 files changed, 4 insertions(+)
create mode 100644 models/cnn_model.pt
create mode 100644 models/xgboost_model.json
create mode 100644 outputs/m5_models/model_comparison.png
create mode 100644 outputs/m5_models/model_results.csv
create mode 100644 outputs/m5_models/xgb_confusion_matrix.png
To https://github.com/ShivaNagaVaraPrasadBathula/AAI590-Capstone-Group4-RHD-Mu
Detection.git
3653b86..bab7d8b main -> main

Push: [] code=0
[] https://github.com/ShivaNagaVaraPrasadBathula/AAI590-Capstone-Group4-RHD-Mu
Detection
```

□ Notebook 4 Complete

Models saved:

- `models/xgboost_model.json` — XGBoost classifier
- `models/cnn_model.pt` — CNN state dict
- `models/imputer.pkl` — feature imputer

Outputs:

- `outputs/m5_models/model_results.csv` — performance comparison table
- `outputs/m5_models/model_comparison.png` — ROC curves + training history
- `outputs/m5_models/xgb_confusion_matrix.png` — XGBoost confusion matrix

Next: Module 6 Notebook 5 — Hyperparameter Tuning + SHAP Analysis

AAI590 Capstone | Group 4 | University of San Diego | 2026

```
In [12]: import pandas as pd
from pathlib import Path

results_df = pd.read_csv('/content/model_outputs/model_results.csv')
print('— Model Comparison Summary —')
print(f'{"Metric":<15} {"XGBoost":>10} {"CNN":>10}')
print('-' * 37)
for _, row in results_df.iterrows():
```

```

pass # already printed above

# Reprint cleanly
results_df = results_df.set_index('model').T
print(results_df.to_string())
print()
print(f'Primary model (XGBoost) AUC : {results_df["XGBoost"]["auc_roc"]:.4f}')
print(f'Comparison model (CNN) AUC : {results_df["CNN"]["auc_roc"]:.4f}')
print(f'Better model by AUC : {"XGBoost" if results_df["XGBoost"]

```

```

— Model Comparison Summary —
Metric          XGBoost          CNN
-----
model           XGBoost          CNN
accuracy        0.556250    0.493750
auc_roc         0.596099    0.500000
f1_score        0.512027    0.661088
sensitivity     0.471519    1.000000
specificity     0.638889    0.000000

Primary model (XGBoost) AUC : 0.5961
Comparison model (CNN) AUC : 0.5000
Better model by AUC      : XGBoost

```

In []: